Who submitted the workload?

How to reproduce the tests and what is still missing | 16 October 2026

Enable it in the exporter

This adds `submitted_by` to `karta_pod_workload_info` and `karta_workload_status`. Karta's CRD and SDK stay unchanged. Admission captures direct Kubernetes submissions. CEL reads the metadata supplied by each platform.

```
exporter:
  enabled: true
  useCatalog: true
  attribution:
    admission:
      enabled: true
    submittedByExpression: |
      (
        [
          "workload.dgxc.nvidia.com/submitter-id",
          "run.ai/submitted-by",
          "workflow.example.com/submitter",
          "platform.example.com/username",
          "platform.example.com/created-by"
        ]
        .filter(key, key in annotations && annotations[key] != "")
        .map(key, annotations[key])
        +
        [
          "identity.example.com/submitted-by",
          "osmo.submitted_by",
          "workflow.example.com/submitter",
          "platform.example.com/username",
          "platform.example.com/created-by"
        ]
        .filter(key, key in labels && labels[key] != "")
        .map(key, labels[key])
        + [""]
      )[0]
```

Admission uses the Karta operator's TLS webhook. Keep the operator and webhook enabled. Label only the direct-submission test namespace; the default capture scope is Deployments, Jobs and Pods. Set `admission.enabled: false` for extraction only.

Direct Kubernetes submission

Download the Job and Deployment request files into your working directory.

Create and label the test namespace as an administrator:

```
kubectl create namespace karta-label-direct
kubectl label namespace karta-label-direct \
  karta.run.ai/capture-submitter=true
```

Use a user's kubeconfig with permission to create/patch Jobs and Deployments in that namespace. The test used a one-hour client certificate for `alice@example.test`, without impersonation.

```
kubectl --kubeconfig "$USER_KUBECONFIG" auth whoami
kubectl --kubeconfig "$USER_KUBECONFIG" apply \
  -f "direct-job-request.json"
kubectl --kubeconfig "$USER_KUBECONFIG" apply \
  -f "direct-deployment-request.json"
```

The Deployment ran. The Job completed after 300 seconds. Its annotation was `workload.dgxc.nvidia.com/submitter-id: alice@example.test`. The Pod inherited the value in the metric without copying the annotation.

```
karta_workload_status{
  workload="cel-label-job", phase="Completed",
  submitted_by="alice@example.test"
} 1
karta_pod_workload_info{
  workload="cel-label-job", submitted_by="alice@example.test"
} 1
```

Try supplying another identity at creation, then changing the recorded identity:

```
KEY=workload.dgxc.nvidia.com/submitter-id
jq --arg key "$KEY" '
  .metadata.name="cel-label-forged" |
  .metadata.annotations[$key]="bob@example.test"
' "direct-job-request.json" |
  kubectl --kubeconfig "$USER_KUBECONFIG" create -f -

kubectl --kubeconfig "$USER_KUBECONFIG" -n karta-label-direct \
  annotate job cel-label-job "$KEY=bob@example.test" --overwrite
```

Both were rejected: creation conflicted with the authenticated caller; the update changed an immutable field. Prefilling the caller's own matching identity is allowed. Unrelated labels and annotations are preserved.

This protection covers the selected resources and namespaces. It does not authenticate arbitrary platform metadata. System-created owned children are skipped, and a Kubernetes service account does not reveal the human behind it.

Direct submission and rejection evidence includes the original requests and both rejection results.

Run:AI

Download the training and inference request files into your working directory.

Set `RUNAI_API`, `RUNAI_TOKEN`, `PROJECT_ID` and `CLUSTER_ID`. The test logged in through `/api/v1/token` using a password grant. Use a token with permission to submit into your project.

```
for kind in training inference; do
  jq --arg p "$PROJECT_ID" --arg c "$CLUSTER_ID" \
    '.projectId=$p | .clusterId=$c' \
    "runai-$kind-request-template.json" > "$kind.json"
done

curl -fsS "$RUNAI_API/api/v1/workloads/trainings" \
  -H "Authorization: Bearer $RUNAI_TOKEN" \
  -H 'Content-Type: application/json' --data-binary @"training.json"
curl -fsS "$RUNAI_API/api/v1/workloads/inferences" \
  -H "Authorization: Bearer $RUNAI_TOKEN" \
  -H 'Content-Type: application/json' --data-binary @"inference.json"
```

Both requests use BusyBox, 50m CPU and 64Mi memory, with no GPU request. Training sleeps for 300 seconds. Inference serves HTTP on port 8080 with one replica.

Submitted	Result	Metadata used by CEL
cel-label-training	Job -> Pod; Job completed	Job annotation run.ai/submitted-by=test@run.ai
cel-label-inference	Knative Service -> controllers -> Pod; Ready and HTTP 200	Service has the same annotation; Pod has no submitter key

```
karta_workload_status{
  workload="cel-label-training", phase="Completed",
  submitted_by="test@run.ai"
} 1
karta_pod_workload_info{
  workload="cel-label-inference", submitted_by="test@run.ai"
} 1
```

Extra setup used: Knative had `kubernetes.podspec-schedulername` and `kubernetes.podspec-affinity` enabled. A separate exporter watched the test namespace. Its Role grants `get/list/watch` on `run.ai/workloadrunners`. These are required setup in this lab, not an external identity mapper.

The HTTP check used the Knative revision proxy through a port-forward. External ingress and real model inference were not tested.

Current gap: the value has not been checked against the agreed NVIDIA username. No submitter propagation bug was found in these two Run:AI paths. Root inheritance already covers the missing inference Pod annotation. Suggested follow-up: agree the identity format with Run:AI and document the exporter RBAC/Knative prerequisites in Karta.

Run:AI metric capture contains both workload results.

OSMO

Download the single-task and group request files into your working directory.

Set `OSMO_API` to the gateway and `OSMO_PLATFORM` to a configured platform with CPU capacity. This reproduces the development-auth test. Production authentication must use the installation's normal login; these headers are not a production identity check.

```
for kind in single group; do
  jq --arg platform "$OSMO_PLATFORM" \
    '.file |= gsub("<audit-platform>"; $platform)' \
    "osmo-$kind-request-template.json" |
  curl -fsS "$OSMO_API/api/pool/default/workflow" \
    -H 'Content-Type: application/json' \
    -H 'x-osmo-user: alice@example.test' \
    -H 'x-osmo-roles: osmo-user' --data-binary @-
done
```

The requests submit one task and a leader/worker group. Each task uses Ubuntu 24.04, one CPU, 512Mi memory and a 300-second command. Both workflows completed; all three tasks in the repeated run exited with code 0.

```
Workflow API: submitted_by=alice@example.test
Task Pod:     osmo.submitted_by=alice
CEL reads:    labels["osmo.submitted_by"]
Karta root:   each task Pod, not the whole OSMO workflow
```

```
karta_pod_workload_info{
  workload="e5d1abe4ce374717-e923b371aa294f01",
  workload_kind="Pod", submitted_by="alice"
} 1
```

`karta_workload_status` carried the same `submitted_by="alice"`. Samples were captured while the tasks were running.

Extra setup used: OSMO 6.3.1, development identity headers and a separate audit platform using the existing scheduler. Platform selection was explicit. No service copied submitter metadata or queried the workflow API for the exporter.

Current OSMO bugs: the label drops the email domain, and the one-character user `a` produced no submitter label. Different emails can become the same `alice`.

For now, treat `alice` as a shortened name, not a unique user. Recovering the full identity requires a workflow API lookup, which this PoC does not implement. Suggested OSMO issue: preserve the full principal in an annotation and fix the one-character case.

Current Karta/OSMO gap: there is no Kubernetes workflow root in this setup. Today the metric represents each task Pod. Suggested integration issue: provide a workload-root mapping so one workflow can contain all its tasks.

OSMO outcomes show completion and task exit codes. The original identity edge tests cover the shortened and missing values.

NVCF / NVCT

Download the function, deployment and task request files into your working directory.

This used the self-managed NVCF stack and local service credentials issued through OpenBao. Set API URLs, a management token, and function/task API keys from your installation. Register a compute pool and edit `gpu`/`instanceType` in the deployment and task requests to match it.

```
curl -fsS "$NVCF_API/v2/nvcf/functions" \
  -H "Authorization: Bearer $NVCF_ADMIN_TOKEN" \
  -H 'Content-Type: application/json' \
  --data-binary @"nvcf-function-request.json" > "function.json"
FID=$(jq -r '.function.id' "function.json")
VID=$(jq -r '.function.versionId' "function.json")
DEPLOY="$NVCF_API/v2/nvcf/deployments/functions/$FID/versions/$VID"
curl -fsS "$DEPLOY" -H "Authorization: Bearer $NVCF_ADMIN_TOKEN" \
  -H 'Content-Type: application/json' \
  --data-binary @"nvcf-deploy-request.json"
```

Wait for ACTIVE. For the local invocation gateway, set `NVCF_INVOKE_URL` to its base URL and `NVCF_INVOKE_DOMAIN` to its configured domain without the function ID:

```
curl -fsS "$NVCF_INVOKE_URL/" \
  -H "Host: $FID.$NVCF_INVOKE_DOMAIN" \
  -H "Authorization: Bearer $NVCF_API_KEY"

curl -fsS -X DELETE "$DEPLOY" \
  -H "Authorization: Bearer $NVCF_ADMIN_TOKEN"
curl -fsS "$NVCT_API/v1/nvct/tasks" \
  -H "Authorization: Bearer $NVCT_API_KEY" \
  -H 'Content-Type: application/json' \
  --data-binary @"nvct-task-request.json"
```

The Nginx function returned HTTP 200. After undeploying it, the finite task reached COMPLETED at 100%. The supplied task writes NVCT progress and disables result uploads. NGC was used only for image pulls.

```
karta_pod_workload_info{
  workload="0-sr-df24444a-8807-4fc9-89c6-534c7a701643",
  workload_kind="Pod", submitted_by=""
} 1
```

Both paths had empty `submitted_by` on Pod and workload metrics. The Pod and ICMSRequest exposed account/workload IDs, but no human submitter. The owner link lacked `controller: true`, so Karta selected the Pod as the workload.

Extra setup used: helper overrides `nvcf-worker-init-oss:1.2.1` and `nvcf-worker-task-oss:1.1.4`; the tested chart's image references did not resolve. The two test containers were resized from 1 CPU to 100m to fit the lab. No external identity mapper ran. Hosted NVCF and optional telemetry were not validated.

Current gaps and suggested issues:

- NVCF: keep `submitted_by` empty unless an external audit lookup can recover the original user. A service key alone is not enough. Suggested issue: pass that user into workload/Pod annotations and agree creator versus deployer.
- Karta/NVCF: add a function-level workload mapping. Today, each Pod is a workload.
- NVCT: fix helper image references. Today, use the overrides above.
- NVCA: fix cancellation of malformed requests. One left an ICMSRequest behind and needed manual cleanup after checking its owned resources. The underlying bug still needs a fix.

Execution evidence contains ACTIVE/COMPLETED results and raw metrics. The NVCF issue draft describes the metadata request. No issues have been opened.